

Claude Code

세션 관리 & 프로젝트 변경 이력 관리

Session · Fork · Checkpoint · Context · JSONL · Hook 자동화

대화 전체 기록 / 2026년 7월 2일

1. Session(세션)의 정확한 정의와 의미

Q claude code에서 session의 정확한 정의와 의미가 뭐야?

A

Claude Code에서 “세션”은 하나의 터미널 인스턴스에서 시작해서 종료될 때까지의 대화 흐름 전체를 의미합니다.

핵심 정의

- claude 명령어를 실행하면 새 세션이 시작됨
- 세션 안에서 주고받는 모든 메시지(사용자 프롬프트 + Claude 응답 + 도구 호출 기록)가 하나의 컨텍스트로 누적됨
- 세션은 ~/.claude/projects/<project-id>/ 아래에 JSONL 파일로 저장됨 (대화 히스토리 전체)

세션의 경계

동작	세션에 미치는 영향
/clear	현재 세션의 컨텍스트를 비움 (새 세션처럼 시작, 하지만 같은 프로세스)
/compact	세션은 유지하되 컨텍스트를 요약·압축
터미널 종료	세션 종료, 단 히스토리는 디스크에 남음
claude --continue / --resume	이전 세션을 다시 불러와 이어감
claude --resume <session-id>	특정 세션 ID로 정확히 복귀

왜 중요한가 (GD RDM 프로젝트 관점에서)

세션은 컨텍스트 윈도우의 생명주기 단위이기 때문에:

- 세션이 길어질수록 토큰 소비 누적 → /compact 나 /clear 타이밍 판단에 직결

- session-report 같은 플러그인은 보통 이 세션 단위 JSONL 을 파싱해서 요약·리포트 생성
- 하나의 2018 작업 단위 2019(예: Feature 2 병합 로직 개발)를 세션 하나로 묶어 관리하면 나중에 --resume 으로 그 맥락만 정확히 복귀 가능

CLAUDE.md가 '프로젝트 차원의 영구 기억'이라면, session은 '그 프로젝트 안에서 일어나는 한 번의 작업 대화'라고 보면 정확합니다.

2. 세션 목록·관리 명령어

Q 어 알려줘

A

세션 목록/관리

```
# 현재 디렉토리(프로젝트)의 세션 목록 보기
claude --resume

# 특정 세션 ID로 직접 재개
claude --resume <session-id>

# 가장 최근 세션 이어가기 (대화형 선택 없이 바로)
claude --continue
```

--resume만 입력하면 대화형으로 세션 목록이 뜨고(날짜, 첫 메시지 미리보기 포함), 화살표로 골라서 들어갈 수 있습니다.

세션 안에서 쓰는 슬래시 명령어

명령어	동작
/clear	컨텍스트 초기화 (세션 ID는 유지, 새 작업 시작할 때)
/compact	컨텍스트 압축 요약 (긴 작업 중간에 토큰 절약)
/resume	세션 안에서 다른 세션으로 전환

명령어	동작
/cost	현재 세션의 토큰/비용 확인

세션 파일 직접 확인

```
# Windows
dir %USERPROFILE%\Claude\projects

# 프로젝트별 세션 JSONL 파일들이 여기 쌓임
```

각 JSONL 파일명이 곧 session-id고, 그 안에 메시지·도구호출·타임스탬프가 전부 기록되어 있습니다. session-report 플러그인 쓸 때 이 경로를 참조하게 됩니다.

GD RDM 프로젝트에 적용한다면

```
cd C:\Claude\projects\GD
claude --continue # 어제 하던 Feature 2 작업 이어서
```

새 단계(예: Feature 3 분석 로직) 시작할 땐 /clear로 컨텍스트 비우고 깨끗하게 시작하는 게 토큰도 아끼고 헷갈리지도 않습니다. CLAUDE.md에 진행 상황만 잘 기록해 두면 /clear 해도 다음 세션이 맥락을 금방 따라잡을 수 있습니다.

3. /fork — 세션 분기

Q /fork로 세션을 분기하는건 어떻게 하는거고 어떻게 활용할 수 있어?

A

/fork 가 하는 일

지금까지의 대화 히스토리(컨텍스트) 전체를 복사해서, 원본은 그대로 둔 채 새 세션 ID로 독립적인 분기를 만드는 명령어입니다. 세션을 포크하면 파일시스템이 아니라 대화 히스토리가 분기되며, 포크된 에이전트가 파일을 수정하면 그 변경은 실제로 적용되어 같은 디렉토리에서 작업하는 다른 세션에도 보이게 됩니다. 즉, 컨텍스트는 분기되지만 파일은 공유됩니다.

사용법 두 가지

1) 세션 안에서 바로

```
/fork
```

2) CLI에서 재개하면서

```
claude --resume <session-id> --fork-session
claude --continue --fork-session # 가장 최근 세션을 포크
```

claude --continue --fork-session은 기존 대화의 독립적인 분기를 만듭니다. 같은 컨텍스트, 새로운 경로입니다. git branch와 비슷하지만 세션을 위한 것이라고 보면 됩니다.

언제 쓰면 좋은가

시나리오	설명
위험한 리팩토링 전 체크포인트	위험한 리팩토링을 시도하기 전에 안전하게 실험할 수 있는 분기를 만드는 용도
A/B 프롬프트 테스트	같은 마스터 세션을 두 번 포크해서 두 분기가 갈라지게 두면, 결과의 차이가 컨텍스트 드리프트가 아니라 순수하게 접근 방식의 차이에서 나온다는 것을 보장할 수 있음
버그 디버깅 중 다른 해법 시도	버그를 디버깅 중인데 Claude가 제안한 수정안과 다른 경로를 시도하고 싶을 때, 기존 작업을 잃을 위험 없이 포크해서 시도

GD RDM 프로젝트에 적용한다면

예를 들어 Feature 3 분석 로직에서 두 가지 알고리즘(예: pandas groupby 방식 vs numpy 벡터화 방식)을 비교하고 싶을 때:

```
# 현재 세션 그대로 두고
/fork
# → 분기된 세션에서 numpy 방식 시도
# 원본 세션으로 돌아가면 pandas 방식 그대로 살아있음
```

주의: 파일은 공유되므로, 두 분기를 동시에 같은 디렉토리에서 돌리면 서로의 파일

수정이 충돌할 수 있습니다. 진짜 격리된 실험을 원하면 *git worktree*로 별도 디렉토리까지 분리하는 것이 안전합니다.

4. Checkpoint — 파일 스냅샷과 /rewind

Q checkpoint는 뭐고 어떻게 활용하는거야?

A

Checkpoint 란

Claude가 파일을 수정하기 직전마다 자동으로 만드는 스냅샷입니다. Claude Code는 모든 편집 전에 코드를 자동으로 스냅샷 찍으며, 별도 설정 없이 기본적으로 활성화되어 있습니다. 체크포인트는 Git 커밋이 아니라, Claude Code가 각 파일 편집 전에 자동으로 만드는 세션 단위 스냅샷입니다.

핵심 제약: 무엇이 기록되고 안 되는가

File checkpointing은 Write, Edit, NotebookEdit 도구를 통한 파일 수정만 추적하며, Bash 명령어(echo > file.txt, sed -i 등)를 통한 변경은 체크포인트 시스템에 잡히지 않습니다.

GD RDM 프로젝트에서 Claude가 python rdm_merge.py 같은 bash 스크립트를 실행해서 결과 파일을 생성/덮어쓰면, 그건 체크포인트로 안 잡힙니다.

불러오는 방법

Esc Esc (Esc 두 번 연속)
또는
/rewind

복원 옵션 3 가지

옵션	효과	언제 쓰나
Code only (코드만)	파일만 되돌리고 대화는 유지	실행은 실패했지만 Claude의 이해(맥락)는 맞을 때

옵션	효과	언제 쓰나
Conversation only (대화만)	코드 변경은 유지한 채 Claude 의 컨텍스트만 리셋	코드는 맞는데 그 이후 대화가 산으로 갈 때
Both (둘 다)	파일과 메시지를 모두 그 체크 포인트로 되돌림	가장 흔히 쓰는, 가장 안전한 선택

추가로 "Summarize from here"라는 옵션도 있는데, 이건 그 시점 이후 대화를 AI가 요약해서 압축하는 것으로 파일은 건드리지 않고 컨텍스트 윈도우 공간만 절약해 줍니다.

/fork 와의 차이

구분	/fork	checkpoint(/rewind)
대상	대화 컨텍스트 분기	파일 + (선택적으로) 대화 상태 복원
결과	원본·분기 둘 다 살아있음 (병렬)	과거 시점으로 되돌림 (선형)
용도	A/B 두 접근을 동시에 살려두고 비교	잘못된 길로 갔을 때 되돌리기

GD RDM 프로젝트 적용 팁

- rdm_merge.py 로직을 수정하기 전 한 번 더 확인하고 싶으면 → 그냥 진행하고 잘못되면 /rewind 로 Both 복원
- 대용량 CSV 처리 스크립트를 bash 로 직접 실행해서 파일을 덮어쓰는 경우는 체크포인트가 못 잡으니, 중요한 단계 전엔 git commit 을 따로 해두는 것이 안전합니다

/rewind는 git의 대체재가 아니라 보조 안전망 정도로 생각하면 됩니다.

5. 컨텍스트(Context)란

Q 컨텍스트는 뭐야?

A

Claude Code에서 컨텍스트(Context)는 현재 세션에서 Claude가 "알고 있는" 모든 정보의 총합입니다.

구체적으로 무엇이 포함되나

- 사용자가 입력한 메시지 전체
- Claude 가 응답한 내용 전체
- 도구 호출 기록 (파일 읽기, bash 실행 결과 등)
- CLAUDE.md 같은 시스템 프롬프트

이 모든 것이 하나의 긴 텍스트 덩어리로 Claude에게 전달됩니다.

컨텍스트 윈도우란

Claude가 한 번에 처리할 수 있는 컨텍스트의 최대 크기를 컨텍스트 윈도우라고 합니다. 토큰(token) 단위로 측정되며, 세션이 길어질수록 누적된 대화가 이 한계에 가까워집니다.

세션 시작

↓ 메시지 주고받을수록

컨텍스트 점점 쌓임

↓ 한계에 도달하면

오래된 내용이 밀려나거나 → /compact로 압축 필요

세션과의 관계

개념	설명
세션	대화의 생명주기 단위 (JSONL 파일로 디스크에 저장)
컨텍스트	세션 안에서 현재 Claude가 메모리에 올려두고 있는 내용

세션은 디스크에 영구 저장되지만, 컨텍스트는 현재 활성화된 메모리 공간입니다. --continue로 세션을 재개하면 이전 JSONL을 읽어서 컨텍스트를 복원하는 방식입니다.

6. 턴(Turn)이란

Q Claude.ai에서 이렇게 내가 하나 질문을 하고 그 답변을 받고 하는것 하나 하나는 뭐라고 하는거야?

A

각각을 "턴(Turn)"이라고 합니다.

정확한 용어 정리

용어	설명
턴(Turn)	사용자 메시지 1개 + Claude 응답 1개의 쌍
메시지(Message)	사용자 또는 Claude가 보낸 개별 발화 하나
대화(Conversation)	여러 턴이 쌓인 전체 흐름

예시로 보면

[턴 1]
 사용자: "세션이 뭐야?"
 Claude: "세션은 ..."

[턴 2]
 사용자: "컨텍스트는 뭐야?"
 Claude: "컨텍스트는 ..."

[턴 3]
 사용자: "이걸 워드로 저장해줘"
 Claude: "..."

Claude API 관점에서도 동일하게 턴이라는 단위를 사용하며, 컨텍스트 윈도우 한계도 "몇 턴까지 기억할 수 있는가"로 체감하게 됩니다.

7. Claude Code 컨텍스트 윈도우 한계

Q Claude code의 컨텍스트 윈도우 한계는 얼마야?

A

모델별 컨텍스트 윈도우 (2026 년 6 월 기준)

모델	컨텍스트 윈도우	최대 출력
Claude Fable 5	1,000,000 토큰 (1M)	128,000 토큰
Claude Opus 4.8	1,000,000 토큰 (1M)	128,000 토큰
Claude Sonnet 4.6	1,000,000 토큰 (1M)	64,000 토큰
Claude Haiku 4.5	200,000 토큰 (200K)	-

1M 토큰이 실제로 어느 정도냐면

- 약 75 만 단어 분량의 영문 텍스트
- 장편 소설 10 권 분량
- 5 만 줄짜리 Python 프로젝트 전체가 들어가는 크기

그래도 생각보다 빨리 찰 수 있습니다

코딩 세션에서는 수십 개의 파일을 읽고, 빌드를 실행하고, 여러 차례 리팩토링을 거치다 보면 2시간짜리 세션만으로도 현실적으로 10만~30만 토큰을 소비할 수 있습니다.

한계에 가까워지면 어떻게 되나

컨텍스트가 가득 차면 하드 에러가 나기 전에 먼저 품질 저하 증상이 나타납니다.

- Claude 가 이전에 결정한 내용을 잊어버림
- 서로 모순된 응답을 함
- 앞서 논의한 함수명이나 파일명을 잘못 말하는 현상 발생

GD RDM 프로젝트 관점에서

Claude Code는 컨텍스트 윈도우가 얼마나 찼는지 실시간으로 표시해 줍니다. 새 작업으로 전환할 때는 `/clear`, 작업 도중 계속 이어가야 할 때는 `/compact`를 사용하는 것이 기본 원칙이며, Claude Code는 한계에 가까워지면 자동으로 압축(auto-compact)도 해줍니다.

542컬럼 스키마 + CSV 파일 내용 + 코드가 동시에 컨텍스트에 쌓이는 GD RDM 프로젝트는 세션이 길어질수록 토큰 소비가 빠를 수 있으니, Feature 단계가 마무리될 때마다 `/clear`로 정리하는 습관이 중요합니다.

8. 세션 재개 관련 Q&A

Q GD RDM 프로젝트를 하나의 CLI로 작업을 하다가 그 창을 닫고 다시 몇시간 있다가 다시 cmd로 창을 열어 이어서 작업을 하면 이게 하나의 세션이야 아님 다른 세션이 생긴거야?

A

다른 세션이 생긴 것입니다. 터미널 창을 닫으면 세션이 종료되고, 새로 cmd를 열어서 claude를 실행하면 새 세션 ID가 생성됩니다.

정확한 흐름

첫 번째 작업

cmd 창 열기 → claude 실행 → 세션 A (JSONL 파일 생성됨)

... 작업 중 ...

cmd 창 닫기 → 세션 A 종료 (JSONL은 디스크에 보존)

몇 시간 후

cmd 창 새로 열기 → claude 실행 → 세션 B (새 JSONL 파일 생성)

← 세션 A의 컨텍스트는 없음, 백지 상태

이전 작업을 이어가려면

방법 1: 가장 최근 세션을 바로 이어감 (제일 편함)

```
claude --continue
```

방법 2: 목록 보고 골라서 재개

```
claude --resume
```

방법 3: 특정 세션 ID 지정

```
claude --resume <session-id>
```

Q `claude --continue`으로 이어서 하면 그게 하나의 세션이 계속 이어지는거야?

A

맞습니다, 하나의 세션이 계속 이어지는 것입니다. `--continue`는 새 세션을 만드는 게 아니라, 종료됐던 이전 세션의 JSONL 파일을 불러와서 그 컨텍스트 위에서 대화를 재개하는 방식입니다. 세션 ID도 그대로 유지됩니다.

정확한 흐름

첫 번째 작업

claude 실행 → 세션 A 시작

... 작업 ...

cmd 창 닫기 → 세션 A 일시 중단 (JSONL 저장)

몇 시간 후

claude `--continue` → 세션 A 재개 (같은 세션 ID)

... 이전 대화 컨텍스트 그대로 살아있음 ...

cmd 창 닫기 → 세션 A 다시 중단

또 몇 시간 후

claude `--continue` → 세션 A 또 재개

--continue는 가장 최근 세션을 불러오는 것이므로, 그 사이에 실수로 claude를 그냥 실행(인자 없이)했다면 새 세션 B가 "가장 최근"이 되어버립니다. 그럴 땐 claude --resume으로 목록을 보고 세션 A를 직접 골라서 재개해야 합니다.

9. 세션 목록명은 무엇으로 저장되는가

Q `claude --resume` 을 입력하면 세션 목록이 나오는거 같은데 그 목록명은 뭘로 저장되는거야?

A

`claude --resume`을 입력하면 나오는 목록은 별도의 "이름"이 있는 게 아니라, 아래 정보들을 조합해서 표시해 줍니다.

목록에 표시되는 정보

- 날짜/시간 — 세션이 마지막으로 업데이트된 시각
- 첫 번째 사용자 메시지의 앞부분 — 그 세션에서 제일 처음 입력한 메시지가 미리보기 텍스트로 표시됨

```
> 2026-06-20 14:32 "rdm_merge.py에서 542컬럼 병합 로직 작성해줘..."
2026-06-19 10:11 "Feature 3 분석 모듈 시작하자..."
2026-06-18 09:05 "안녕, GD RDM 프로젝트 세팅부터..."
```

실제 저장 파일명 (세션 ID)

내부적으로는 `~/.claude/projects/<프로젝트경로>/` 아래에 UUID 형태의 JSONL 파일로 저장됩니다.

```
~/.claude/projects/C:\Claude\projects\GDW
├── a3f8d2c1-4b5e-7f9a-...jsonl ← 세션 A
├── b7c4e1d0-2a3f-8c1b-...jsonl ← 세션 B
└── ...
```

사용자가 지정하는 세션 이름은 따로 없고, "첫 메시지 미리보기 + 날짜" 조합으로 구분하는 방식입니다. GD RDM 프로젝트처럼 장기 작업을 할 때는 첫 메시지를 "Feature 2 병합 로직 작업 시작" 처럼 명확하게 입력해 두면 나중에 목록에서 찾기가 훨씬 편합니다.

10. `/clear` 를 굳이 해야 하는가

Q /clear 로 정리하면 내가 이어서 계속 작업할때 위의 내용을 다시 알려줘야 하는 거 아니야?

A

네, 맞습니다. /clear를 하면 컨텍스트가 초기화되기 때문에 Claude는 이전 대화 내용을 전혀 모르는 상태가 됩니다. 그래서 CLAUDE.md가 핵심 해결책입니다.

CLAUDE.md의 역할

CLAUDE.md는 세션이 시작될 때마다 자동으로 컨텍스트에 로드됩니다. /clear를 하거나 새 세션을 시작해도 CLAUDE.md는 항상 살아있습니다.

```
/clear 실행
↓
대화 히스토리는 사라짐
↓
하지만 새 세션 시작 시 CLAUDE.md 자동 로드
↓
Claude가 프로젝트 맥락을 즉시 파악
```

정리

구분	대화 히스토리	CLAUDE.md
/clear 후	사라짐	살아있음
새 세션 시작 후	없음	자동 로드됨
--continue 후	복원됨	자동 로드됨

Q 그럴거면 왜 굳이 /clear를 해야 하는거야? 그냥 다 찰때까지 계속 쓰다 auto compact되면 마칠때까지 그냥 기다리면 되는거야 아냐?

A

맞는 말씀입니다. 사실 꼭 /clear를 해야 하는 건 아닙니다. 다만 /clear를 권장하는 이유가 있습니다.

Auto-compact 에만 맡겼을 때의 문제

- 중요한 결정 사항이 요약되면서 디테일이 날아갈 수 있음
- 압축 타이밍을 내가 통제 못함 — 작업 한가운데서 갑자기 발생할 수 있음
- 비용 낭비 — 필요 없는 과거 대화(이미 끝난 Feature 1 내용 등)가 계속 토큰을 잡아먹음

언제 /clear 가 진짜 유용하냐면

"Feature 2 완료 → Feature 3 시작" 처럼 명확히 작업 단위가 바뀔 때입니다.

현실적인 결론

상황	권장 방법
같은 작업을 계속 이어갈 때	그냥 두거나 /compact
새 Feature·새 작업으로 전환할 때	/clear
컨텍스트가 거의 찼는데 작업이 안 끝났을 때	/compact

CLAUDE.md만 잘 관리되어 있다면 /clear 후에도 맥락 복원이 빠르기 때문에 두려워할 필요가 없습니다. 하지만 귀찮으시면 auto-compact에 맡겨도 크게 문제는 없습니다. 다만 비용과 품질 면에서 살짝 손해를 볼 수 있다는 정도입니다.

11. 프로젝트 변경 이력 관리 개선안

Q 개인적으로 프로젝트의 크기가 커지면서 중요한 내용을 놓치지 않기 위해서 1. dialog.txt: 전체 대화 내용을 그대로 저장, 2. session_log.txt: 세션별 주요 사항 저장, CLAUDE.md: 이건 Claude code가 알아서 저장하고 프로젝트 폴더의 변경사항은 Git으로 추적/저장하고 있어. 그런데 매번 주요 작업 후 이것들을 일일이 업데이트 해달라고 하는 것도 귀찮고 그렇다고 안하면 중요한 내용이 누락될 수 있는 위험성도 있어. 이런 상황에서 어떻게 하면 프로젝트의 변경 사항들을 효율적으로 관리할 수 있을지 검토하고 개선안을 제안해 줘

A

현재 방식의 문제점

파일	문제
dialog.txt	전체 대화를 수동 저장 → 토큰 낭비 + 누락 위험
session_log.txt	매번 수동 요청 필요 → 귀찮음 + 일관성 없음
CLAUDE.md	Claude가 알아서 업데이트하지만 타이밍이 불규칙
Git	코드 변경은 잘 잡히지만 "왜 바꿨는지" 맥락은 없음

핵심 문제는 "수동 요청"에 의존하고 있다는 점입니다.

개선안: Hook 기반 자동화

Claude Code의 Hook 기능을 활용하면 수동 요청 없이 자동으로 기록할 수 있습니다. Hook은 특정 이벤트(파일 저장, 세션 종료, 도구 호출 등)가 발생할 때 자동으로 스크립트를 실행하는 기능입니다. 컨텍스트 윈도우 바깥에서 실행되므로 토큰 소모가 없습니다.

1) dialog.txt → 폐기 권장

세션 JSONL 파일이 이미 전체 대화를 저장하고 있습니다. 별도로 dialog.txt를 유지하는 건 중복입니다.

```
~/claude/projects/GD/<session-id>.jsonl
→ 이게 곧 dialog.txt입니다
```

필요할 때 JSONL을 파싱해서 읽으면 되므로 dialog.txt는 없애도 됩니다.

2) session_log.txt → Hook 으로 자동화

세션 종료 시(PostSession Hook) 자동으로 핵심 사항을 추출해서 기록하도록 합니다.

```
// .claude/settings.json
{
  "hooks": {
```



```

"PostSession": [
  {
    "matcher": "",
    "hooks": [{
      "type": "command",
      "command": "python scripts/save_session_log.py"
    }]
  }
]
}
}

```

```

# scripts/save_session_log.py
# 세션 종료 시 자동 실행
# - 오늘 날짜/세션 ID
# - 수정된 파일 목록 (Git diff로 추출)
# - 마지막 N개 메시지 요약
# → session_log.txt에 자동 append

```

3) CLAUDE.md → Hook + Git commit 메시지로 보완

CLAUDE.md는 Claude가 업데이트하되, Git commit 시점을 CLAUDE.md 업데이트 트리거로 활용합니다.

작업 완료

- git commit
- PostToolUse Hook 실행
- CLAUDE.md의 "현재 진행 상황" 섹션 자동 업데이트

4) Git commit 메시지를 구조화

feat(Feature3): 분석 모듈 이상값 탐지 로직 추가

- 542컬럼 중 수치형 컬럼 대상 Z-score 기반 탐지
- 임계값 설정 가능하도록 config.yaml에 파라미터 추가
- 다음 작업: 웹 대시보드 연동

최종 제안 구조

작업 중

└─ Claude가 파일 수정 → Git이 자동 추적

작업 마무리 시 (하루 1회 정도)

└─ git commit (구조화된 메시지)

└─ Hook → CLAUDE.md "진행 상황" 섹션 자동 갱신

└─ Hook → session_log.txt 자동 append

필요할 때만

└─ JSONL 파싱으로 전체 대화 열람 (dialog.txt 불필요)

토큰 관점에서의 이득

작업	기존	개선 후
dialog.txt 저장	매번 Claude에게 요청 → 토큰 소모	JSONL 그대로 활용 → 0 토큰
session_log 업데이트	매번 Claude에게 요청 → 토큰 소모	Hook 스크립트 → 0 토큰
CLAUDE.md 업데이트	수시로 요청 → 토큰 소모	commit 시 Hook → 최소화

결론 — 적용 우선순위

- 즉시 — dialog.txt 폐기, JSONL 로 대체
- 단기 — git commit 메시지 구조화 (템플릿 만들어 두기)
- 중기 — PostSession Hook 으로 session_log.txt 자동화
- 중기 — CLAUDE.md 자동 갱신 Hook 연결

12. JSONL 이란

Q JSONL이 뭐야?

A

JSONL(JSON Lines)은 JSON 객체를 한 줄에 하나씩 저장하는 텍스트 파일 형식입니다.

일반 JSON vs JSONL 비교

일반 JSON — 전체가 하나의 덩어리

```
[
  {"role": "user", "content": "안녕"},
  {"role": "assistant", "content": "안녕하세요!"}
]
```

JSONL — 한 줄에 JSON 하나씩

```
{"role": "user", "content": "안녕"}
{"role": "assistant", "content": "안녕하세요!"}
```

Claude Code 가 JSONL 을 쓰는 이유

대화가 진행되면서 한 줄씩 계속 append(추가)하면 되기 때문입니다. 일반 JSON은 새 내용을 추가할 때마다 파일 전체를 다시 써야 하지만, JSONL은 맨 아래에 한 줄만 추가하면 되므로 대화 기록처럼 계속 누적되는 데이터에 매우 적합합니다.

실제 Claude Code 의 세션 JSONL 구조

```
{"type": "user", "content": "세션이 뭐야?", "timestamp": "2026-06-20T14:32:01Z"}
{"type": "assistant", "content": "세션은 하나의 터미널...", "timestamp": "2026-06-20T14:32:05Z"}
{"type": "tool_use", "tool": "read_file", "path": "rdm_merge.py", "timestamp": "2026-06-20T14:33:10Z"}
{"type": "user", "content": "컨텍스트는 뭐야?", "timestamp": "2026-06-20T14:35:22Z"}
```

정리

항목	내용
확장자	.jsonl
구조	한 줄 = JSON 객체 하나
장점	빠른 append, 대용량 처리에 유리
Claude Code에서의 역할	세션 전체 대화 기록 저장소

Q 이 파일이 모든 대화를 다 저장하고 있어?

A

네, 저장하고 있습니다. 다만 몇 가지 중요한 특성이 있습니다.

저장되는 것

- 사용자가 입력한 모든 메시지
- Claude 의 모든 응답
- 도구 호출 기록 (파일 읽기, bash 실행, 결과값 등)
- 각 메시지의 타임스탬프
- 토큰 사용량 등 메타데이터

주의할 점

- /clear 를 하면 JSONL 도 분리됩니다 — /clear 이후 대화는 내부적으로 새 블록으로 구분될 수 있음
- /compact 를 하면 압축된 요약으로 대체됩니다 — 압축 전 원본 대화가 JSONL 에서 요약문으로 치환될 수 있어서 압축 이전의 세부 내용이 날아갈 수 있음
- Bash 명령어 결과는 일부 잘릴 수 있습니다 — 출력이 너무 긴 경우 전체가 아닌 일부만 저장될 수 있음

Q compact 시 압축된다는 이게 바로 문제야 내가 dialog.txt를 이용하는 이유야**A**

맞습니다. 정확히 핵심을 짚으셨습니다.

문제의 본질

```

/compact 실행
↓
JSONL의 이전 대화 내용이 요약문으로 치환됨
↓
"A 방법을 시도했다가 실패해서 B로 바꿨다"는 맥락이
"B 방법을 사용함" 한 줄로 압축될 수 있음
↓
나중에 "왜 B를 썼지?" 를 추적할 수 없게 됨
  
```

특히 GD RDM처럼 장기 프로젝트에서는 시행착오의 맥락이 매우 중요한데, 이게 compact로 날아가면 나중에 같은 실수를 반복하거나 왜 그런 결정을 했는지 알 수 없게 됩니다.

그래서 dialog.txt 방식이 맞습니다

구분	JSONL	dialog.txt
compact 후	이전 내용 손실	전부 보존
형식	기계가 읽기 좋은 형식	사람이 읽기 좋은 형식
검색	파싱 필요	텍스트 검색 바로 가능
누적	자동	수동 요청 필요

dialog.txt 자체가 나쁜 게 아니라 "매번 수동으로 요청해야 한다"는 점이 문제입니다. Hook 자동화가 여기서 진가를 발휘합니다. 매 턴이 끝날 때마다 Hook으로 대화 내용을 자동으로 dialog.txt에 append하면, 수동 요청 없이도 한 글자도 빠짐없이 누적 저장이 가능합니다.